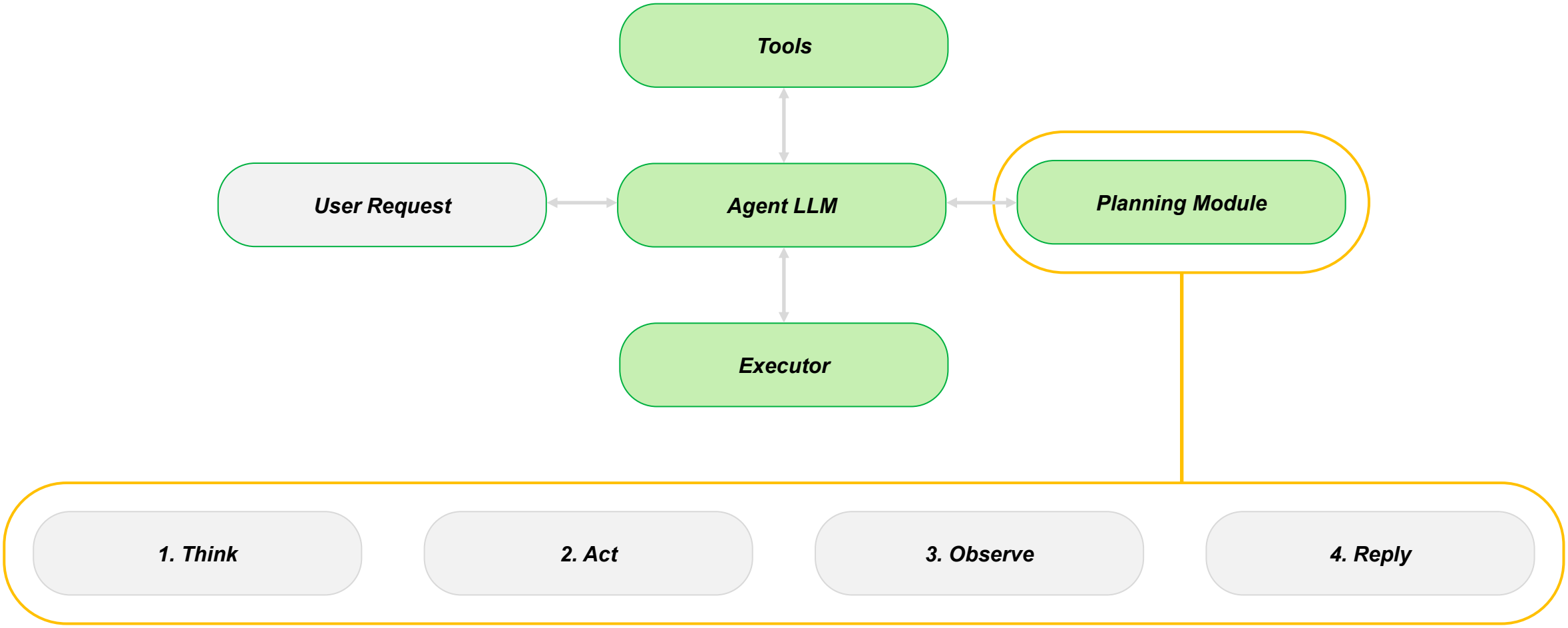
REPLY – LUISS PROJECTS

Roma, 2026

REPLY

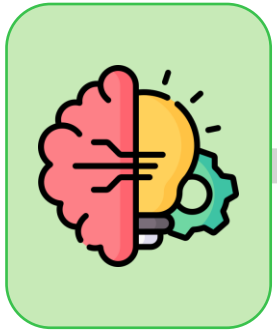
WHY AGENTS?

AGENTS ARE DEFINED BY ACTING



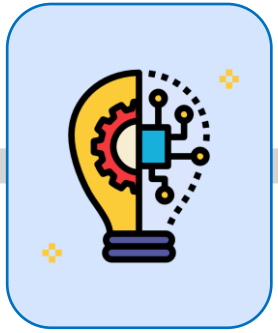
KEY COMPONENTS

LLMs

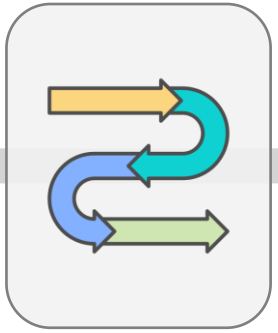


Memory

Tools



Chains

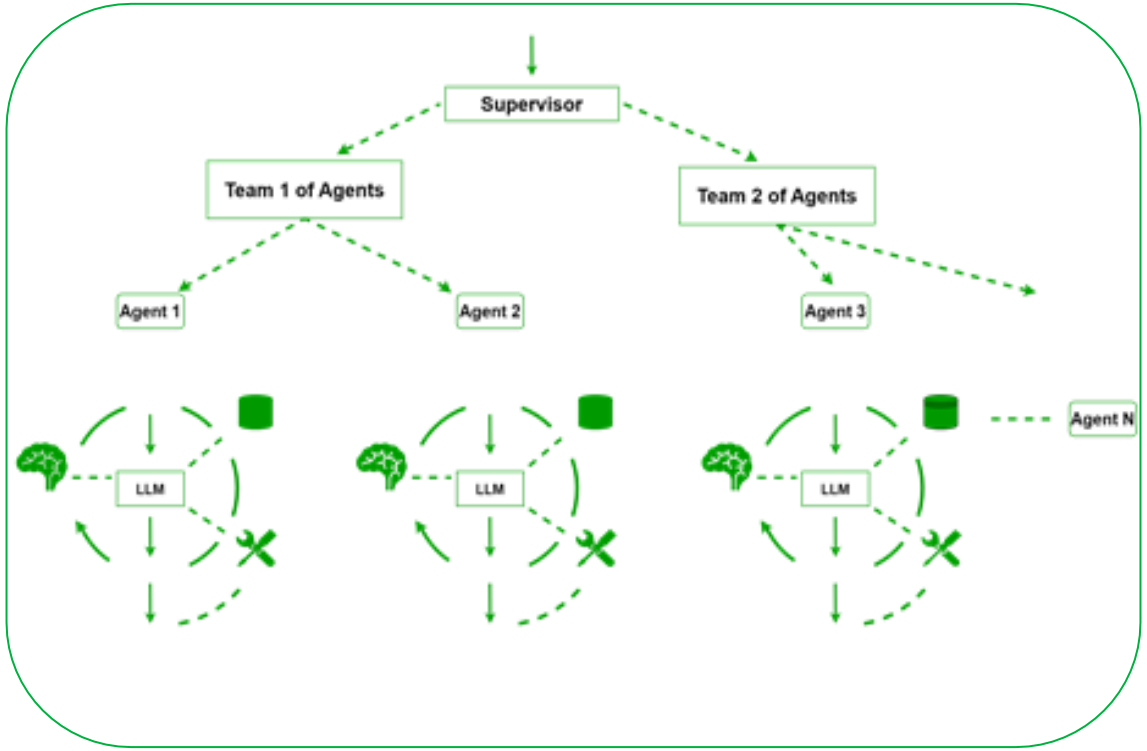


Prompt Engineering

An agent combines **LLMs** for reasoning, **Memory** for context, **Tools** to interact with external data, **Chains** for sequential executions, and **Prompt Engineering** to optimize responses.

MULTI-AGENT SYSTEM

A multi-agent system is composed of **multiple interacting agents**, each with unique capabilities, working collectively to achieve complex goals



Key **features**:

- **Interactivity**: agents converse, decide together
- **Diversity**: each agent brings a specialized skill set for varied tasks
- **Collaborative Goals**: agents share common targets
- **Efficiency**: divide tasks, speeds up work

1. AGENTS FOR DATA QUALITY

REPLY

CONTEXT

The client NoiPA

NoiPA is the **digital platform** of the Ministero dell'Economia e delle Finanze that manages salaries, timesheets, and tax/social security obligations for employees of the Italian Public Administration.

It allows users to view payslips and annual tax certifications online, update personal information, and manage administrative and HR-related procedures.

! NoiPA periodically receives datasets from heterogeneous sources containing demographic, economic, or health data. Currently, validation is manual or nonexistent.

NoiPA Data sources

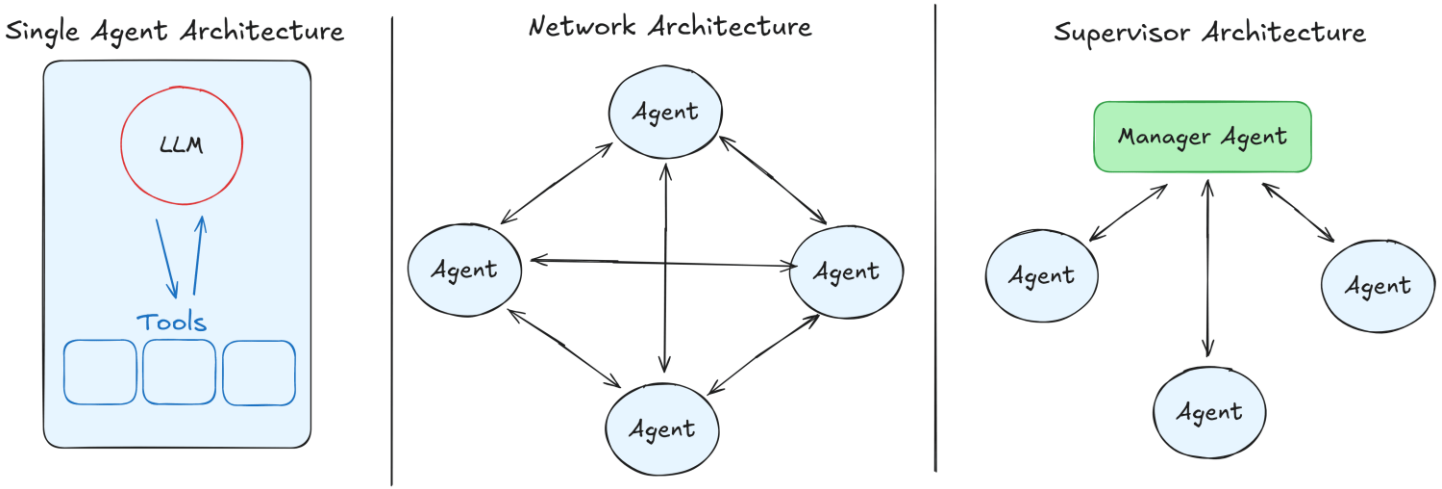
CSV

JSON

DBs

GOAL

Build a **multi-agent system** that receives a raw dataset and automatically fix the dataset and produces a quality report including anomalies, correction suggestions, and a reliability score.



DATASET

Example

Rata	Ente	Descrizione	Tipo imposta	Imposta	Spesa
202512	*12*	*Ministero dell'interno*	*Erariali*	*Addizionali Comunali*	20163.25
202512	*10*	*Ministero della giustizia*	*Previdenziali*	*IRAP*	14163.89
202512	*11*	*Ministero dell'interno*	*Varie*	*IRPEF*	13888.00
202512	*10*	*Ministero del lavoro*	*Erariali*	*Addizionali Regionali*	14163.89

Input data

CSV file

You should test your solution with more than one dataset.

AGENTS

Schema Validation

1 Data Type Validation

Checks that each column matches the expected data type (e.g. a date column doesn't contain free text).

2 Naming Convention Check

Validates column names against naming standards (casing, special characters, reserved words).

Completeness Analysis

1 Null / Missing Value Detection

Counts and locates null, empty, or placeholder values (e.g. "N/A", "-", "unknown") per column and per row.

2 Completeness Rate Calculation

Computes a completeness percentage per column and overall for the dataset.

3 Sparse Column Detection

Identifies columns that are almost entirely empty and may be candidates for removal or investigation.

Consistency Validation

1 Cross-Column Checking

Validates logical relationships between columns (es. birth date is coherent with age).

2 Format Consistency

Ensures that values follow consistent formats within the same column (e.g. dates are all in DD/MM/YYYY).

3 Duplicate Detection

Identifies exact and near-duplicate records based on key columns (e.g. same tax code appearing twice).

Anomaly

AGENTS

eness
sis

Consistency Validation

1 Cross-Column Checking

Validates logical relationships between columns (es. birth date is coherent with age).

2 Format Consistency

Ensures that values follow consistent formats within the same column (e.g. dates are all in DD/MM/YYYY).

3 Duplicate Detection

Identifies exact and near-duplicate records based on key columns (e.g. same tax code appearing twice).

Anomaly Detection

1 Univariate Outlier Detection

Applies statistical methods to detect values that fall far outside the expected range for numerical columns.

2 Categorical Anomaly Detection

Flags unexpected or rare categorical values (e.g. a job title that appears only once out of thousands of records).

Remediation

1 Correction Suggestions

For each issue, the agent proposes a possible remediation action (e.g. "fill nulls with median", "remove duplicate record at row 4521", "correct date format").

OUTPUT

The output of the project should be the **report with data quality results**.

Code on GitHub

PPT presentation

A *suggestion* for the final presentation is to use **Streamlit** to create a simple graphical interface to display the final output.

2. CLASSICAL VS MULTI-AGENT

REPLY

CONTEXT

The client

Border control authorities and airport operators manage large volumes of passenger transits daily, each associated with rich metadata: timestamp, gate, route, nationality, document type, control outcome, and security alerts.

! Currently, anomaly detection is largely reactive. A **proactive analytical system** could identify suspicious patterns before they escalate into operational risks or security incidents.

Client data sources

CSV

JSON

DBs

GOAL

Implement the same anomaly detection system **twice:** once with a classical pipeline and once with a multi-agent architecture

Produce a **comparative analysis** to argue which approach is more convenient and under what operational conditions.

DATASET

Example

Anno	Mese	Aeroporto	Descrizione Aeroporto	Città	Paese	Occorrenze
2025	*Aprile*	*366*	*Aeroporto di Fiumicino*	*Roma*	*Italia*	*90*
2025	*Febbraio*	*948*	*Aeroporto di Malpensa*	*Milano*	*Italia*	*20000*
2025	*Dicembre*	*1273*	*Aeroporto di Fiumicino*	*Roma*	*Italia*	*45679*
2025	*Settembre*	*1538*	*Aeroporto di Malpensa*	*Milano*	*Italia*	*56211*

Input data

CSV file

IMPLEMENTATION - CLASSICAL

Feature Engineering → **Data Preparation** → **Anomaly Detection**

Feature engineering via SQL joins across all relational tables (transit volumes, non-conformity rates, alert rates per gate/route/nationality/document type)

Historical baseline construction using statistical methods (rolling averages, seasonal decomposition)

Anomaly detection using IsolationForest, LOF, or Z-score on engineered features

Post Processing → **Output**

Rule-based post-processing layer (e.g. "alert rate on route X exceeds 3x baseline")

Ranked list of anomalous patterns with scores and static report

IMPLEMENTATION - AGENTS

Data Agent → **Baseline Agent** → **Outlier Detection Agent**

Queries the DB based on user-defined perimeter

Builds historical baselines per gate, route, nationality, and document type using rolling averages and seasonal decomposition

Applies IsolationForest, LOF, or Z-score on the engineered features

Risk Profiling Agent → **Report Agent**

Applies business rules equivalent to the classical post-processing layer (e.g. "alert rate on route X exceeds 3x baseline").

Uses an LLM to produce a dynamic Transit Anomaly Report where each flagged pattern is accompanied by a natural language explanation of the anomaly.

OUTPUT

The output of the project should be a **transit anomaly report** with results.

Code on GitHub

PPT presentation

A *suggestion* for the final presentation is to use **Streamlit** to create a simple graphical interface to display the final output.

Q&A

REPLY

c.cantelmo@reply.it

https://www.linkedin.com/in/carlotta-cantelmo-b63681215/

s.granato@reply.it

https://www.linkedin.com/in/simone-granato/

REPLY